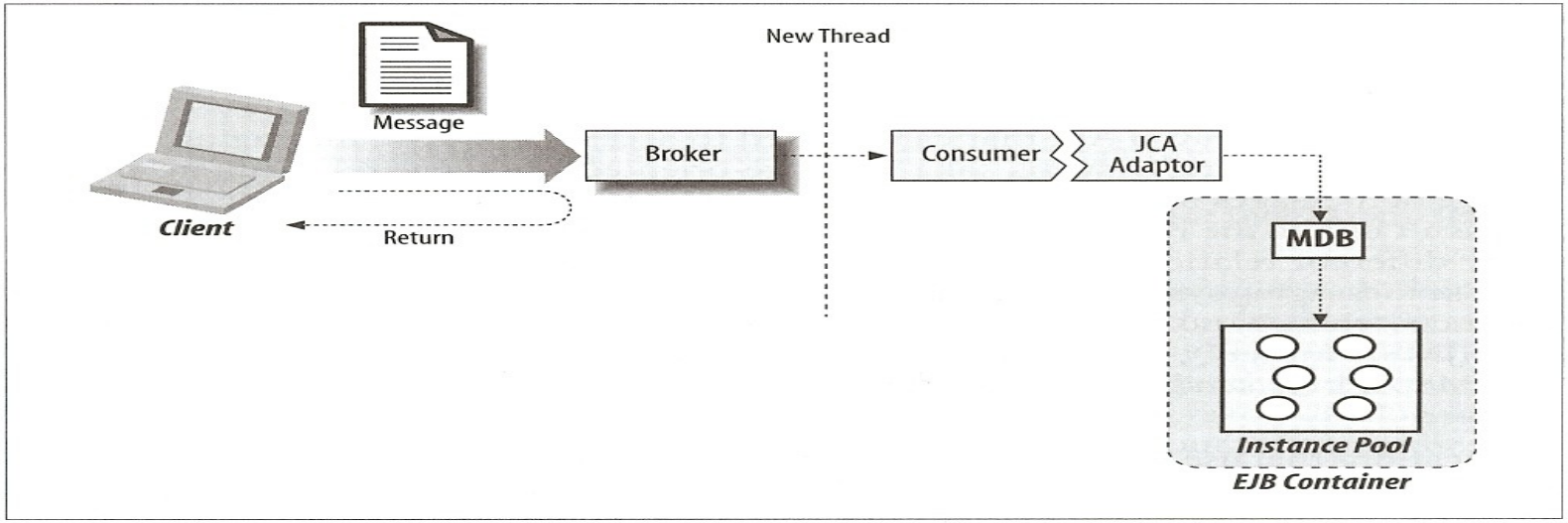
Message-Driven Beans

Chapter 8

Overview

- Message-driven beans are asynchronous listeners
- Clients send request to messaging system and go about their business



JMS and Message-Driven Beans

- All EJB 2.0 (and later) vendors must support a Java Message Service (JMS) provider
  - EJB 2.1 (and later) supports any messaging service complaint with Java Connector Architecture (JCA)
- Some JMS provider is essential for supporting Message-Driven Beans

JMS as a Resource

- JMS is vendor-neutral API for accessing enterprise messaging systems
  - Similar to JDBC and databases
  - Example messaging systems: IBM MQSeries, BEA WebLogic JMS, Sun ONE Message Queue, JbossMQ, JBoss HornetQ
  - Wikipedia list 16 JMS providers
- Jakarta has tutorials on JMS
  - see tutorial in Java EE 9 tutorial ch 48,49

JMS Terminology

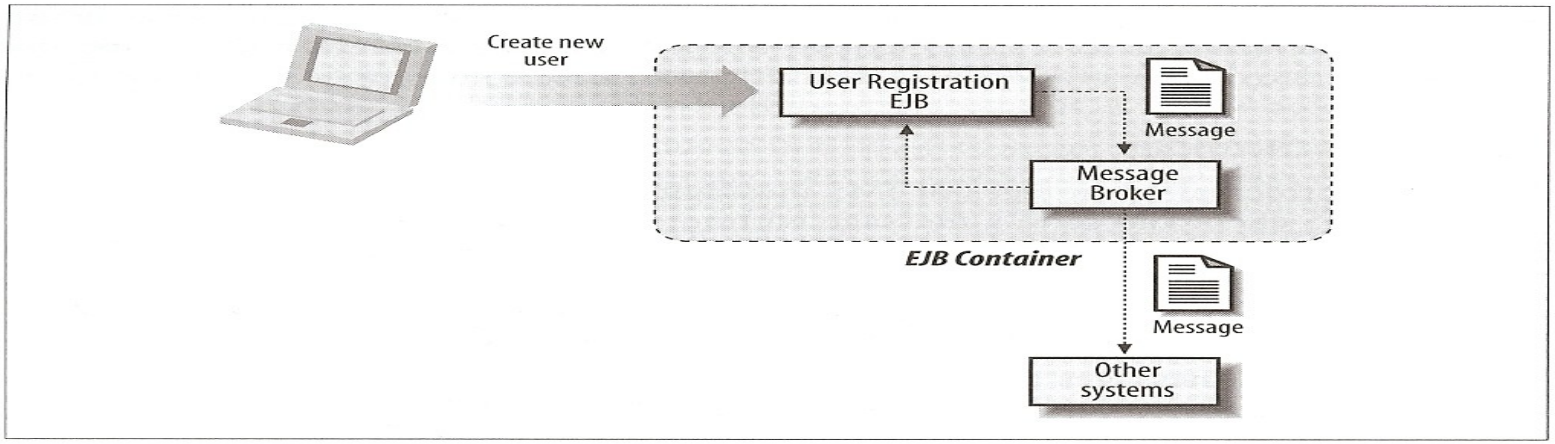
- Applications using JMS are *JMS clients*
- Messaging system is called a *JMS provider*
- *JMS Application* is business system with one or more JMS clients and a provider
- JMS client that sends a message is called a *producer*
- JMS client that receives messages is *consumer*
- All EJBs can be producers
- Java applications or message-driven beans can be consumers

JMS is Asynchronous

- Primary advantage of JMS messaging
  - Client can send a message without waiting for a reply
- Clients send message to JMS destination
  - Topic or Queue
- Consumer of messages will get message from JMS provider
- More convenient than RMI as means for distributing system across multiple platforms

Example

- Suppose UserRegistrationEJB wants to notify systems of new user
  - Without being coupled to them
- Can send JMS message indicating new user and other systems can pick up the message



What is Happening Here?

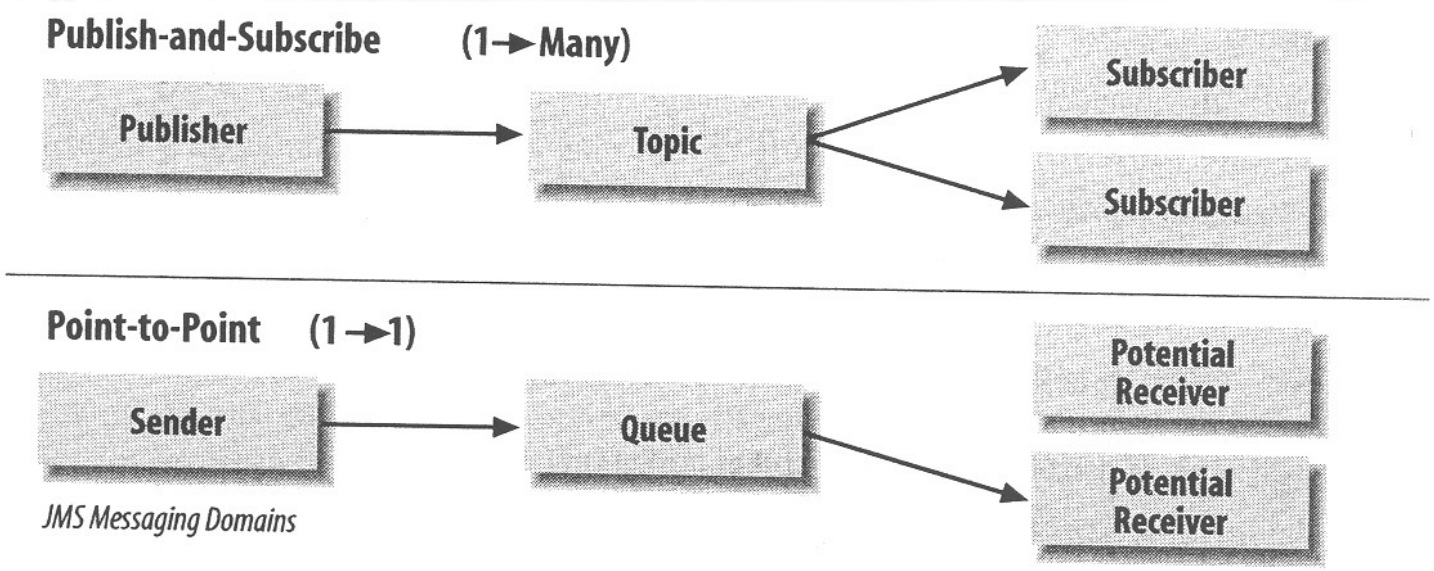
- To send JMS message, need
  - Connection to JMS provider
  - Destination address for message
- Applications receiving messages may be
  - Message-driven beans
  - Other applications in the enterprise
  - Applications in other organizations who need to process the message
- Any JMS client can subscribe to a topic and receive messages
  - Will also see *queue* which is point-to-point

Transactions and Security

- Transactions and security contexts of sender not propagated to the receiver
  - Not possible because message provider is in the middle
- Receiver is often in different security domain
  - Would not make sense to propagate security
- Transactions can not be propagated to receiver
  - May be unknown number of receivers
  - Receivers may take time to process messages
- Can involve JMS provider in transaction
  - If transaction fails, provider discards message

JMS Messaging Models

- Two types, each with interfaces/classes
  - Publish-and-subscribe
  - Point-to-point



JMS Messaging Domains

Publish-and-Subscribe

- Producer sends message to a virtual channel
  - Called a *topic*
- Consumers can *subscribe* to a topic
- Messages published to a topic are broadcast to all subscribed consumers
- Customers can use durable subscription
  - Disconnect and later collect messages they missed
- Publish/subscribe used when producer does not care which subscriber processes messages

Point-to-Point

- Sender publishes messages to a *queue*
- Receiver reads messages from the queue
  - *Each* message is consumed by *one* consumer
- To change from publish/subscribe to point-to-point
  - Replace "Topic" with "Queue" when getting Destination
  - Destination is parent class for Topic and Queue
- Point-to-Point used when Sender is communicating with a particular receiver

JMS-Based Message-Driven Beans

- MDBs are
  - Stateless
  - Server-side
  - Transaction-aware components
  - For processing messages delivered via JMS
- Container manages
  - Transactions
  - Security
  - Resources
  - Concurrency and
  - Message acknowledgement
  - Container can create multiple instances to handle high message traffic

@MessageDriven

- MDBs identified with @MessageDriven annotation
  - or <message-driven> element of XML deployment descriptor
- @MessageDriven takes one attribute: activationConfig
  - which takes array of @ActivationConfigProperty annotations
  - Each @ActivationConfigProperty has two attributes:
    - propertyName and
    - propertyValue
- @ActivationConfigProperty describes messaging properties of MDB
  - For JMS includes: acknowledgeMode, messageSelector, destinationType, and subscriptionDurability

Example

```
@MessageDriven(activationConfig={
  @ActivationConfigProperty(
     propertyName="destinationType",
     propertyValue="javax.jms.Queue"),
  @ActivationConfigProperty(
     propertyName="messageSelector",
     propertyValue="MessageFormat='Version 3.4'"),
  @ActivationConfigProperty(
     propertyName="destination",
     propertyValue="queue/reservationQueue")})
```

propertyName= "messageSelector"

- Allows MDB to specify criteria to filter which messages will be received
- Can select based on properties set in message header by clients
  - In previous example, client sets MessageFormat property in header
- Selection uses syntax similar to SQL WHERE clause

propertyName= "acknowledgeMode"

- Whether EJB container should acknowledge receipt of message to JMS provider
  - Do not need to specify in most cases
  - Usually container manages transactions and acknowledgement take place in context of transaction with JMS provider
- Values
  - Auto-acknowledge
    - Means container should acknowledge for each message received, ASAP
  - Dups-ok-acknowledge
    - Means container does not have to acknowledge right away

propertyName= "subscriptionDurablilty"

- Indicates whether the subscription is Durable or NonDurable
- Durable
  - Any messages delivered when MDB is disconnected are delivered when connection is restored
- NonDurable
  - Any messages delivered when MDB is disconnected are lost
- When destination type is Queue, durability is not an issue
  - Messages stay in queue until delivered to a queue listener

propertyName = "destinationType"

- Specifies type of message source
  - for JMS either
    - `javax.jms.Topic` or
    - `javax.jms.Queue`

MessageDrivenContext

- Similar to Session Context
  - extends EJB Context with no additional content
- May be injected with `@jakarta.annotation.Resource`
  - `@Resource MessageDrivenContext context;`
- Only useful methods:
  - `getUserTransaction, getRollbackOnly, setRollbackOnly`
- Other methods not applicable to Message Driven beans
  - throw `RuntimeException` if called
- MDBs also have access to their own ENC
  - Can hold references to environment entries, other EJBs and resources

`MessageListener` Interface

- MDBs usually implement this interface
- Defines the `onMessage(Message m)` method
  - Called to process each message from the configured message system
- This interface is separate from the MessageDrivenBean interface
- Allows separation of MDB from the type of messaging system it is connected to
  - So MDBs can process messages from nonJMS messaging systems

MDB `onMessage()` Method

- This is where all business logic goes
- Container passes messages to MDB by calling `onMessage(Message message)`
- `onMessage` is called by one thread at a time
- onMessage body parses message to know how to handle it
- Can use session or other beans as needed

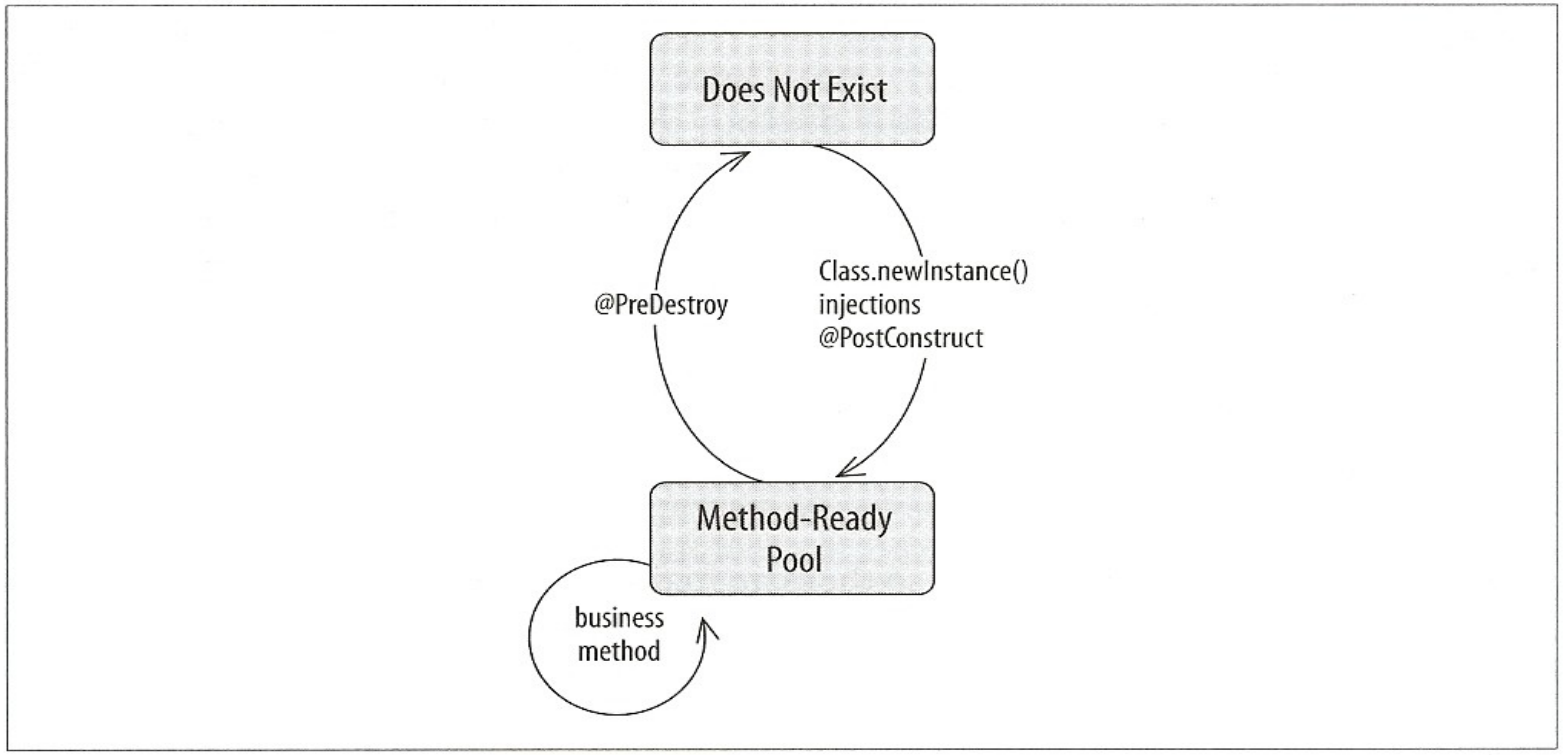
Sending Messages from MDB

- MDB can send messages to a Queue or Topic
  - No different from other JMS clients
- E.g.

```
public void deliverTicket (MapMessage reservationMsg, TicketDO ticket)
    throws JMSException, NamingException {
      Queue queue = (Queue)reservationMsg.getJMSReplyTo();
      //  or  Queue queue = (Queue)
      //new InitialContext().lookup ("queue/ticketQueue");
      Connection connect = connectionFactory.createConnection();
      Session session = connect.createSession(true,0);
      MessageProducer sender = session.createProducer(queue);
      ObjectMessage message = session.createObjectMessage();
      message.setObject(ticket);
      sender.send(message);
      connect.close();
 }
```

Life Cycle of a Message-Driven Bean

- Similar to stateless session beans

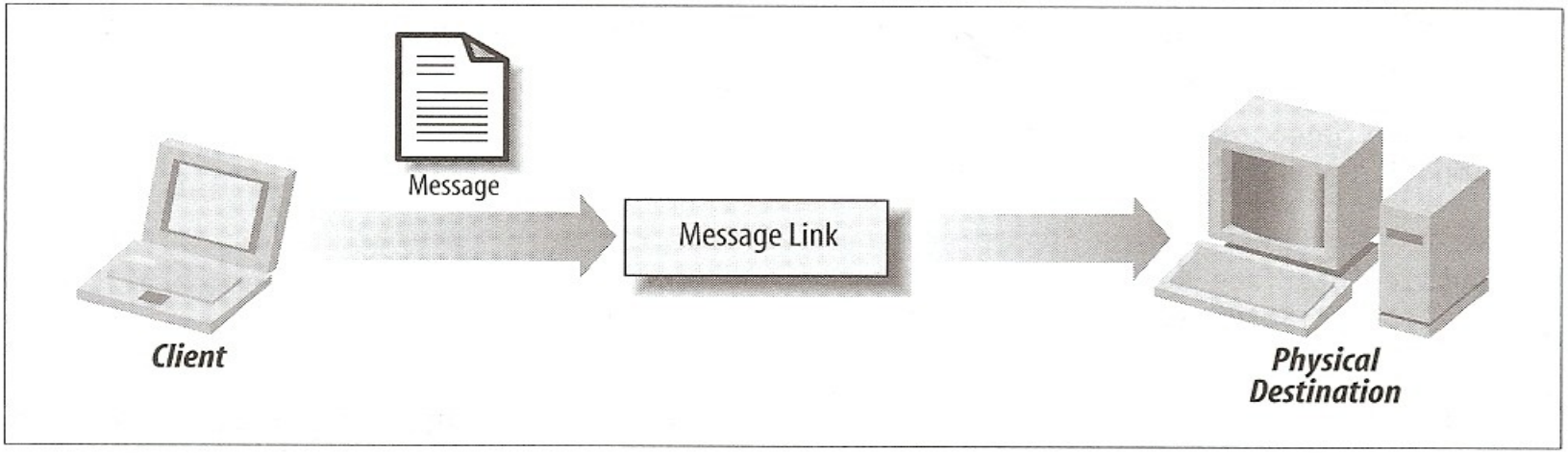


Connector-Based Message-Driven Beans

- MDBs allowed to use other messaging systems besides JMS
- Any messaging system that is based on Java Connector Architecture (JCA 1.6) can be used
  - Provides Service Provider Interface
    - Allows any enterprise information system to plug into any J2EE system
  - MDB then implements messaging interfaced defined by JCA connector

Message Linking

- Allows messages sent from EJBs to be routed to a specific MDB in same deployment



- No big deal – see text

JMS APIs: Connection and Session

- To send message, need connection to JMS provider
  - And destination address for message
- `ConnectionFactory` creates a `Connection`
- `Connection` object used to create `Session`
  - `Session session = connect.createSession(true, 0);`
  - Parameters should be ignored
    - Container manages transaction and acknowledgement mode for JMS resources from ENC
- `Session` used to create `MessageProducer`
  - And messages to publish using `MessageProducer`

Message Types

- Messages have headers and message bodies
  - Header has delivery information and metadata
- Message body has application data
- Several message types
  - `TextMessage, MapMessage, ObjectMessage, StreamMessage, BytesMessage`
  - `MapMessage` map values can be primitive types, strings, arrays or objects with corresponding accessors
    - Use `setInt, getString, setObject`, etc. to set map values

Example: Send JMS Message

- Send string version of a ticket to TicketTopic

```
String ticketDescription = ticket.toString();
@Resource(mappedName=" java:comp/env/jms/TopicFactory")
private ConnectionFactory factory;
@Resource(mappedName="TicketTopic")
private Topic topic;
…
Connection connect = factory.createConnection();
Session session = connect.createSession(true, 0);
MessageProducer producer = session.createProducer(topic);

TextMessage textMsg = session.createTextMessage();
textMsg.setText(ticketDescription);

producer.send(textMsg);
connect.close();
```

JMS Application Client

- Java application can listen to JMS topic
  - using JNDI to find resources
- In order to react to messages arriving, application implements the interface `jakarta.jms.MessageListener`
  - Requires onMessage(Message m) method which is called when message arrives
- Session is used to create Consumer to receive messages
  - Instead of Producer to send messages

Summary

- Message-Driven beans are less tightly coupled than session beans
  - The bean server can be down and not block the client
- They are considered message-oriented middleware (MOM)
- Clients do not need to know they are talking to message-driven beans
  - They just send off a message to the correct queue or topic
- These are good for activities that may take a long time to complete
  - Such as order fulfillment